

GABARITO - Exercícios Aula 15

Aqui está o gabarito completo e detalhado para os 20 exercícios propostos sobre a base de dados Sakila. Este material está estruturado para que você compreenda o "porquê" de cada resposta, reforçando os conceitos ensinados na aula.

Módulo 1: O Universo dos Joins (Junções)

1. O que aconteceria conceitualmente...

Resposta: Ocorrerá o que chamamos de **Produto Cartesiano**. O motor da base de dados combinará todas as linhas da tabela `actor` com todas as linhas da tabela `film`. Se existirem 200 atores e 1000 filmes, o resultado terá 200.000 linhas. Isso geralmente não é útil porque cria correspondências falsas e sem sentido (ex: associar um ator a um filme no qual ele não participou).

2. Utilizando INNER JOIN...

SQL

```
SELECT c.first_name, c.last_name, a.address
FROM customer c
INNER JOIN address a ON c.address_id = a.address_id;
```

3. LEFT JOIN com categorias...

SQL

```
SELECT f.title, fc.category_id
FROM film f
LEFT JOIN film_category fc ON f.film_id = fc.film_id;
```

(Nota: O `LEFT JOIN` garante que, se houver algum filme na tabela `film` sem um registro correspondente na `film_category`, o filme aparecerá na mesma na lista, mas com o `category_id` preenchido a `NULL`.)

4. Sintaxe USING...

SQL

```
SELECT p.payment_id, p.amount, c.first_name -- (ou apenas SELECT *)
FROM customer c
JOIN payment p USING (customer_id);
```

5. A armadilha do NATURAL JOIN...

Resposta: O `NATURAL JOIN` une as tabelas usando **todas** as colunas que possuem exatamente o mesmo nome em ambas. As tabelas `customer` e `staff` partilham não apenas o `store_id`, mas também colunas como `active` e `last_update`. O SQL tentará fazer corresponder os registros em que o `last_update` de um cliente seja idêntico ao `last_update` do funcionário, o que não faz sentido relacional e resultará num cruzamento de dados indesejado ou vazio.

Módulo 2: O Desafio dos Valores Nulos (NULL)

6. Alugueres não devolvidos (return_date nula)...

SQL

```
SELECT * FROM rental
```

```
WHERE return_date IS NULL;
```

7. Alugueres já devolvidos...

SQL

```
SELECT rental_id  
FROM rental  
WHERE return_date IS NOT NULL;
```

8. Lógica ternária e o = NULL...

Resposta: O SQL utiliza a lógica ternária (Verdadeiro, Falso e Desconhecido). O `NULL` significa ausência de valor (Desconhecido). Ao perguntar se "Desconhecido é igual a Desconhecido" (`return_date = NULL`), a resposta do SQL não é "Verdadeiro", mas sim "Desconhecido". Como o filtro `WHERE` apenas exibe as linhas cuja condição seja estritamente "Verdadeira", nenhuma linha será retornada.

9. LEFT JOIN com filtro de Nulos (Filmes sem cópia)...

SQL

```
SELECT f.title  
FROM film f  
LEFT JOIN inventory i ON f.film_id = i.film_id  
WHERE i.inventory_id IS NULL;
```

10. O comportamento do DISTINCT com NULL...

Resposta: Retornará **1 linha**. Embora o sinal de igual não funcione para os Nulos, uma exceção é aberta para o `DISTINCT` (e agrupamentos). O motor agrupa as linhas tratando todos os valores `NULL` como idênticos para não repetir linhas "vazias", exibindo apenas uma única ocorrência de `NULL`.

Módulo 3: Resumindo com Funções de Agregação

11. Quantidade total de filmes...

SQL

```
SELECT COUNT(*) AS total_filmes  
FROM film;
```

12. Receita total da locadora...

SQL

```
SELECT SUM(amount) AS receita_total  
FROM payment;
```

13. Duração média dos filmes...

SQL

```
SELECT AVG(length) AS duracao_media  
FROM film;
```

14. Valores de aluguer mais barato e mais caro...

SQL

```
SELECT MIN(rental_rate) AS valor_minimo, MAX(rental_rate) AS valor_maximo
FROM film;
```

15. Clientes únicos que realizaram pagamentos...

SQL

```
SELECT COUNT(DISTINCT customer_id) AS clientes_unicos
FROM payment;
```

Módulo 4: Agrupamentos e Filtros (GROUP BY e HAVING)

16. Total gasto por cliente...

SQL

```
SELECT customer_id, SUM(amount) AS total_gasto
FROM payment
GROUP BY customer_id;
```

17. O erro da Regra de Ouro (Rigor Técnico)...

Resposta: A consulta gera erro porque a coluna `staff_id` foi solicitada no `SELECT`, mas não está envolvida numa função matemática (como `SUM`) e **não foi declarada no `GROUP BY`**. O SQL empacota os dados por `customer_id`, mas como um cliente pode ter sido atendido por vários funcionários diferentes, a base de dados não sabe qual `staff_id` exibir para representar o grupo.

18. Agrupamento com HAVING (> 150 dólares)...

SQL

```
SELECT customer_id, SUM(amount) AS total_gasto
FROM payment
GROUP BY customer_id
HAVING SUM(amount) > 150;
```

19. Diferença entre WHERE e HAVING na contagem...

SQL

```
SELECT staff_id, COUNT(payment_id) AS total_pagamentos
FROM payment
WHERE amount > 2.00
GROUP BY staff_id;
```

Módulo 5: Ordem Lógica e Execução (Unindo os Mundos)

20. A Ordem Lógica de Processamento (O "Cérebro" do SQL):

- **Passo 1 (FROM/JOIN):** O motor procura as tabelas `category` e `film_category` e faz o cruzamento lógico das linhas onde as chaves correspondem (`c.category_id = fc.category_id`), gerando o conjunto de dados base.

- **Passo 2 (WHERE):** Antes de qualquer cálculo, atua como "triagem" e analisa linha a linha os dados cruzados, eliminando fisicamente aquelas em que a categoria seja 'Action' ou 'Horror'.
- **Passo 3 (GROUP BY):** Pega no que sobrou da triagem e separa (organiza) os registos em "pacotes" baseados na coluna nome da categoria (c.name).
- **Passo 4 (HAVING):** Funciona como "auditor" e conta o número de IDs de filme dentro de cada pacote formado no passo anterior. Se a contagem desse grupo não for maior que 50, o pacote inteiro é descartado.
- **Passo 5 (SELECT):** Finalmente, com os pacotes válidos e os cálculos processados, projeta a informação visualmente para o utilizador com os apelidos definidos (categoria e total_filmes).